

3

Introduction to Classes and Objects

OBJECTIVES

- To learn how to define a class.
- To learn how to create an object.
- To learn how to define member functions.
- To learn how to declare data members.
- To learn how to call a member function.
- To learn how to use a constructor.
- To know how to separate interface from implementation.

3.2 Classes, Objects, Member Functions and Data Members

- What is a class?

- A class is the specification of a data type that you define.

// A class to define a box

class Box ← Class name

{
 ↓ Access-specifier

public:

 int length;

 int width;

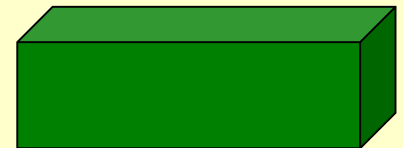
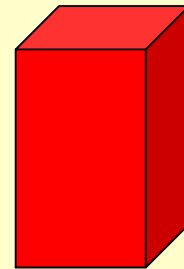
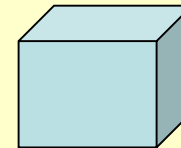
 int height;

};

← Keyword: a word reserved by C

← Data members
(Attributes)

Box



3.2 Classes, Objects, Member Functions and Data Members

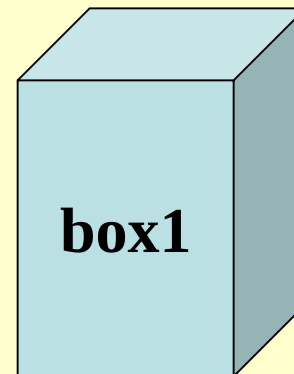
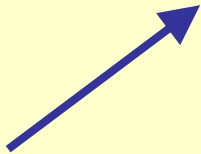
- **What is an object?**

- A reusable software component that is declared (like integer variable) and created inside a class.

```
// A class to define a box
```

```
Box box1; // Create an object defined  
          // by the class Box
```

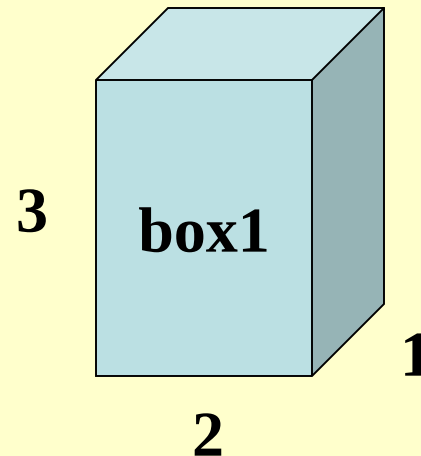
Object name



3.2 Classes, Objects, Member Functions and Data Members

- **What are data members?**
 - They are data items defined in a class.
 - Sometimes we called them attributes.

```
// Define the dimension of a box  
box1.length = 2;  
box1.width = 1;  
box1.height = 3;
```



3.2 Classes, Objects, Member Functions and Data Members

- **What is a member function?**
 - **A member function is a function defined in a class.**

3.2 Classes, Objects, Member Functions and Data Members

```
// A class to define a box
```

```
class NewBox
```

```
{
```

```
public:
```

```
    int length;
```

```
    int width;
```

```
    int height;
```

```
    int volume()
```

```
{
```

```
    return length * width * height;
```

```
}
```

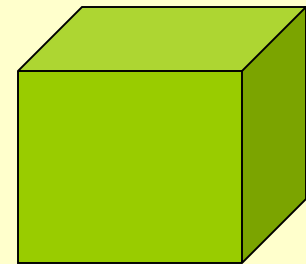
```
};
```

input

parameter
(no input)

member function's

name



data type
of the
returned
value

the statement to be executed
when the member function is called

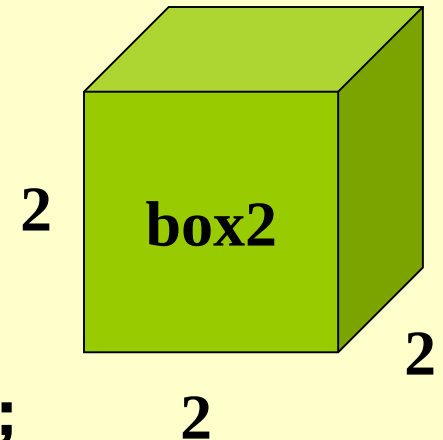
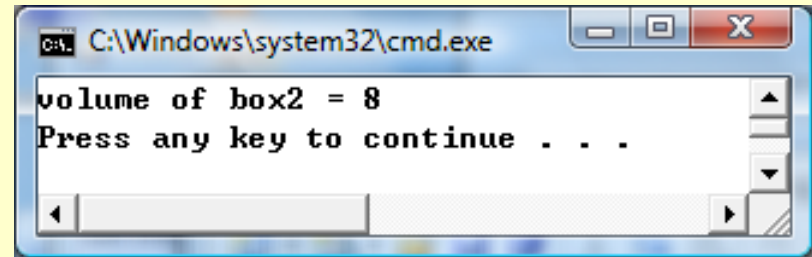
3.2 Classes, Objects, Member Functions and Data Members

```
int main()  
{  
    NewBox box2;
```

```
    box2.length = 2;  
    box2.width = 2;  
    box2.height = 2;  
    cout << "Volume of box2 = ";  
    cout << box2.volume() << endl;  
    return 0;
```

```
}
```

Dot operation – the way to call a member function



3.2 Classes, Objects, Member Functions and Data Members

- Mechanism to execute `box2.volume()`

Execute `box2.volume()`



Find the object `box2` and
then execute its member function `volume`



Execute the statement inside the member
function `volume` and it returns 8

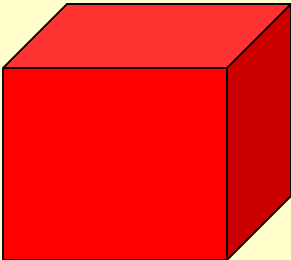


`box2.volume()` returns 8

3.2 Classes, Objects, Member Functions and Data Members

```
// A class to define a box
class NewBox2
{
public:
    int length;
    int width;
    int height;
    int weight(int density)
    {
        return length * width * height * density;
    }
};
```

One input parameter

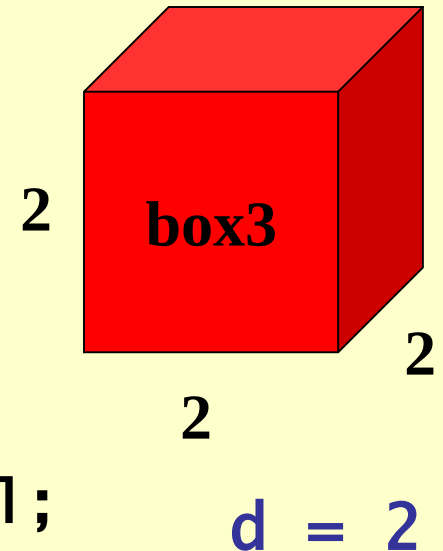
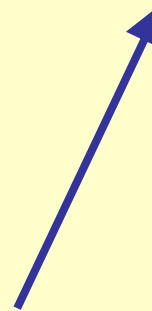
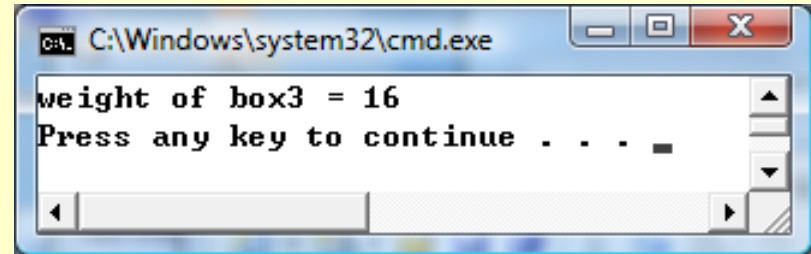


A blue arrow points from the text "One input parameter" to the parameter "density" in the function signature "int weight(int density)". To the right of the code is a 3D red cube with black outlines, representing a box.

3.2 Classes, Objects, Member Functions and Data Members

```
int main()
{
    NewBox2 box3;
    int d = 2;

    box3.length = 2;
    box3.width = 2;
    box3.height = 2;
    cout << "Weight of box3 = ";
    cout << box3.weight(d) << endl;
    return 0;
}
```



3.2 Classes, Objects, Member Functions and Data Members

- Mechanism to execute `box3.weight()`

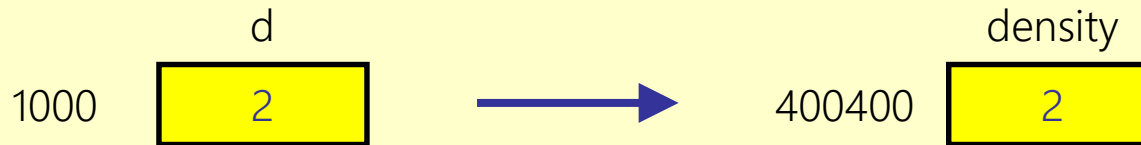
Execute `box3.weight(d)`



Find the object `box3` and then execute its member function `weight` with the input argument `d`

3.2 Classes, Objects, Member Functions and Data Members

Inside the member function `weight` , we have



Execute the statement inside the member function `weight` and it returns 16

`box3.weight(d)` returns 16

3.3 Overview of the Chapter Examples

- **Seven simple examples**
 - Examples used to build a class called **GradeBook**
- **Topics covered:**
 - **Member functions**
 - **Data members**
 - **Constructor**
 - **Separating interface from implementation**

3.3 Overview of the Chapter Examples

- **Example 1: Build up a simple class and an object, and call the member function of the object.**

Outline

fig03_01.cpp

(1 of 1)

```
1 // Fig. 3.1: fig03_01.cpp
2 // Define class GradeBook with a member function displayMessage;
3 // Create a GradeBook object and call its displayMessage function.
4 #include <iostream>
5 using std::cout;
6 using std::endl;
7
8 // GradeBook class definition
9 class GradeBook
10 {
11 public:
12     // function that displays a welcome message to the GradeBook user
13     void displayMessage()
14     {
15         cout << "Welcome to the Grade Book!" << endl;
16     } // end function displayMessage
17 }; // end class GradeBook
18
19 // function main begins program execution
20 int main()
21 {
22     GradeBook myGradeBook; // create a GradeBook object named myGradeBook
23     myGradeBook.displayMessage(); // call object's displayMessage function
24     return 0; // indicate successful termination
25 } // end main
```

```
Welcome to the Grade Book!
```


3.4 Defining a Class With a Member Function

Line 9: `class GradeBook`

- **Class definition**
 - **Keyword `class` followed by the class's name.**

3.4 Defining a Class With a Member Function

```
Line 10: {  
        ...  
Line 17};
```

- **Class body is enclosed in braces ({}).**
 - **Specifies data members and member functions.**

3.4 Defining a Class With a Member Function

Line 11: `public:`

- **Access-specifier `public:`**
 - Indicates that a member function or data member is accessible to other functions and member functions of other classes.

3.4 Defining a Class With a Member Function

Line 13: `void displayMessage()`

- **Member function definition**
 - **Return type of a function**
 - Indicates the type of value returned by the function when it completes its task
 - `void` indicates that the function does not return any value

3.4 Defining a Class With a Member Function

```
Line 22: GradeBook myGradeBook;
```

- **Using a class**
 - A class (e.g., `GradeBook`) is a user-defined type (or programmer-defined type)
 - Can be used to create objects (e.g., `myGradeBook`)

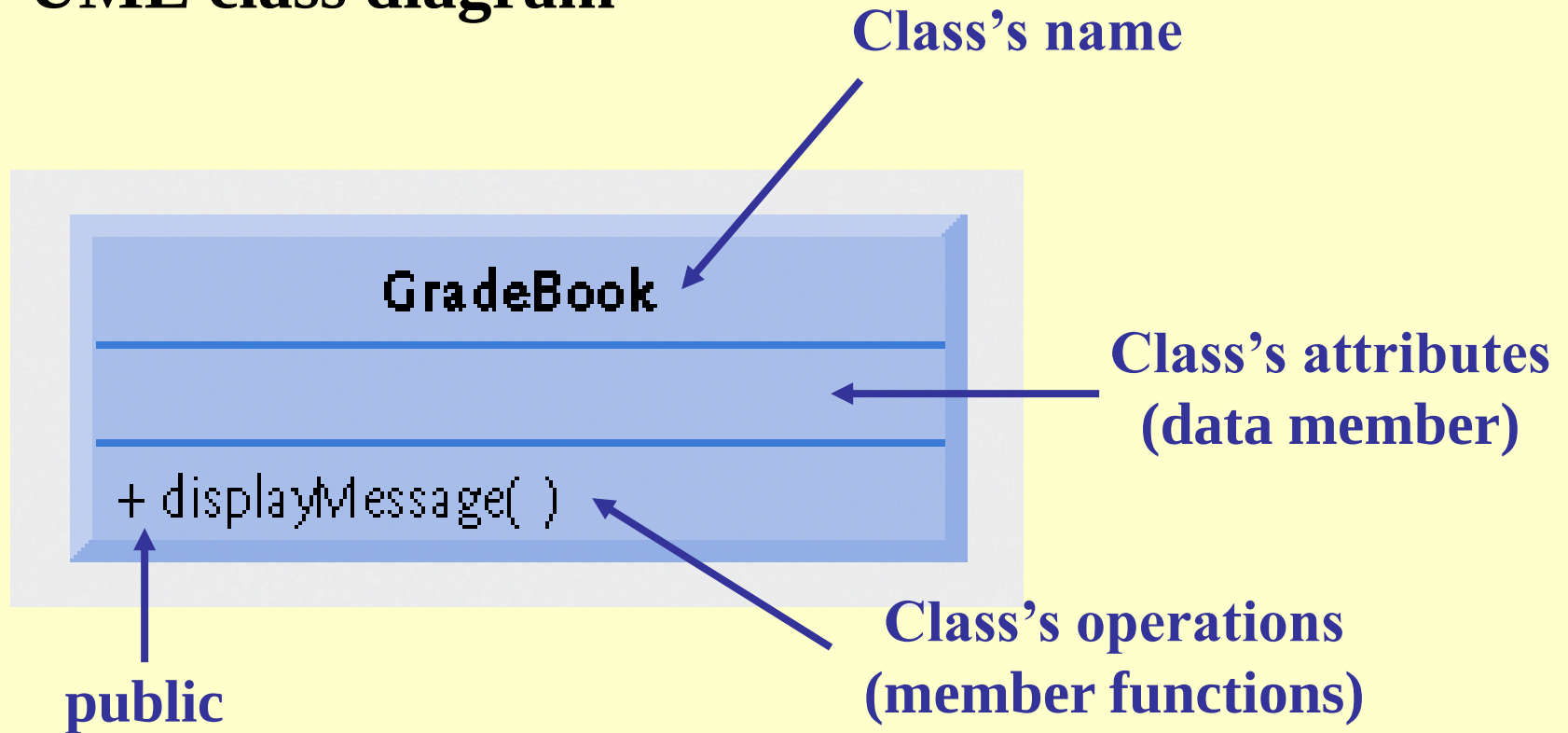
3.4 Defining a Class With a Member Function

```
Line 23: myGradeBook.displayMessage();
```

- **Dot operator (.)** uses to access an object's data members and member functions.

3.4 Defining a Class With a Member Function

- UML class diagram



3.5 Defining a Member Function with a Parameter

- **Example 2: Show a member function with input parameters.**


```

1 // Fig. 3.3: fig03_03.cpp
2 // Define class GradeBook with a member function that takes a parameter;
3 // Create a GradeBook object and call its displayMessage function.
4 #include <iostream>
5 using std::cout;
6 using std::cin;
7 using std::endl;
8
9 #include <string> // program uses C++ standard string class
10 using std::string;
11 using std::getline;
12
13 // GradeBook class definition
14 class GradeBook
15 {
16 public:
17     // function that displays a welcome message to the GradeBook user
18     void displayMessage( string courseName )
19     {
20         cout << "Welcome to the grade book for\n" << courseName << "!"
21             << endl;
22     } // end function displayMessage
23 }; // end class GradeBook
24
25 // function main begins program execution
26 int main()
27 {
28     string nameOfCourse; // string of characters to store the course name
29     GradeBook myGradeBook; // create a GradeBook object named myGradeBook
30

```

Outline

fig03_03.cpp

(1 of 2)

```
31 // prompt for and input course name
32 cout << "Please enter the course name:" << endl;
33 getline( cin, nameOfCourse ); // read a course name with blanks
34 cout << endl; // output a blank line
35
36 // call myGradeBook's displayMessage function
37 // and pass nameOfCourse as an argument
38 myGradeBook.displayMessage( nameOfCourse );
39 return 0; // indicate successful termination
40 } // end main
```

Outline

fig03_03.cpp

(2 of 2)

```
Please enter the course name:
CS101 Introduction to C++ Programming

Welcome to the grade book for
CS101 Introduction to C++ Programming!
```

3.5 Defining a Member Function with a Parameter

```
Line 9: #include <string>
```

- **Header file `string`**
 - Data type `string` represents a string of characters, e.g., “Hello world”.
 - It defines a data type `string` (Line 10) and a library function `getline` (Line 11).

3.5 Defining a Member Function with a Parameter

Line 18: `void displayMessage( string courseName)`

- **courseName** is a function parameter.
 - Additional information needed by a function to perform its task.
 - In this program, **courseName** will be displayed to the screen in the function **displayMessage**.
 - Function may have more than one parameter and parameters should be separated by commas, e.g.,

`void displayMessage(string courseName, int abc)`

3.5 Defining a Member Function with a Parameter

```
Line 33: getline( cin, nameOfCourse);
```

- **Library function `getline`**
 - Used to retrieve input until newline is encountered.
- **In this program, we input a line from standard input into string object `nameOfCourse`.**

3.5 Defining a Member Function with a Parameter

```
Line 38: myGradeBook.displayMessage(nameOfCourse);
```

- **Function argument(s)**
 - **Number, order and types of arguments in a function call must match the number, order and types of parameters in the called function's parameter list.**

3.5 Defining a Member Function with a Parameter

“CS101 Introduction to C++ Programming”

getline (cin, nameOfCourse)
⇒ nameOfCourse = “CS101 ...”

myGradeBook.displayMessage(nameOfCourse);

Call the member function of the object with
nameOfCourse = “CS101 ...”

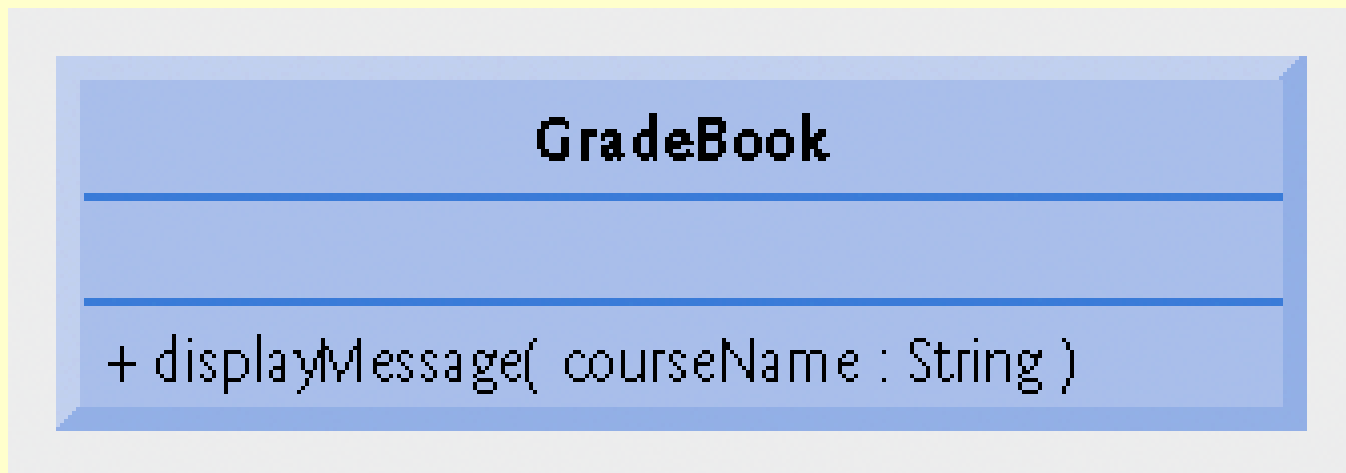
void displayMessage(string courseName)

Display the message with courseName = “CS101 ...”

**“Welcome to the grade book for
CS101 Introduction to C++ Programming”**

3.5 Defining a Member Function with a Parameter

- **Modeled in UML**



3.6 Data Members, *set* Functions and *get* Functions

- **Example 3: Show a data member, and how to use set and get member functions to modify the data member.**

```

1 // Fig. 3.5: fig03_05.cpp
2 // Define class GradeBook that contains a courseName data member
3 // and member functions to set and get its value;
4 // Create and manipulate a GradeBook object with these functions.
5 #include <iostream>
6 using std::cout;
7 using std::cin;
8 using std::endl;
9
10 #include <string> // program uses C++ standard string class
11 using std::string;
12 using std::getline;
13
14 // GradeBook class definition
15 class GradeBook
16 {
17 public:
18     // function that sets the course name
19     void setCourseName( string name )
20     {
21         courseName = name; // store the course name in the object
22     } // end function setCourseName
23
24     // function that gets the course name
25     string getCourseName()
26     {
27         return courseName; // return the object's courseName
28     } // end function getCourseName
29

```

Outline

fig03_05.cpp

(1 of 3)

```

30 // function that displays a welcome message
31 void displayMessage()
32 {
33     // this statement calls getCourseName to get the
34     // name of the course this GradeBook represents
35     cout << "Welcome to the grade book for\n" << getCourseName() << "!"
36         << endl;
37 } // end function displayMessage
38 private:
39     string courseName; // course name for this GradeBook
40 }; // end class GradeBook
41
42 // function main begins program execution
43 int main()
44 {
45     string nameOfCourse; // string of characters to store the course name
46     GradeBook myGradeBook; // create a GradeBook object named myGradeBook
47
48     // display initial value of courseName
49     cout << "Initial course name is: " << myGradeBook.getCourseName()
50         << endl;
51

```

Outline

fig03_05.cpp

(2 of 3)

```
52 // prompt for, input and set course name
53 cout << "\nPlease enter the course name:" << endl;
54 getline( cin, nameOfCourse ); // read a course name with blanks
55 myGradeBook.setCourseName( nameOfCourse ); // set the course name
56
57 cout << endl; // outputs a blank line
58 myGradeBook.displayMessage(); // display message with new course name
59 return 0; // indicate successful termination
60 } // end main
```

Outline

fig03_05.cpp

(3 of 3)

Initial course name is:

Please enter the course name:

CS101 Introduction to C++ Programming

Welcome to the grade book for

CS101 Introduction to C++ Programming!

3.6 Data Members, *set* Functions and *get* Functions

```
Line 38: private:  
Line 39:     string courseName;
```

- **courseName is an Attribute (data member).**
 - Attributes exist throughout the life of the object.
 - Variables in a class definition
 - Each object of class maintains its own copy of attributes.
 - Member functions can use data members first and then data members can be declared later in the class.

3.6 Data Members, *set* Functions and *get* Functions

- **Access-specifier `private`**
 - Makes a data member or member function accessible only to member functions of the class.
 - `private` is the default access for class members.

3.6 Data Members, *set* Functions and *get* Functions

```
Line 19: void setCourseName( string name )  
Line 20: {  
Line 21:     courseName = name;  
Line 22: }
```

- **Software engineering with *set* and *get* functions**
 - **public** member functions that allow clients of a class to set or get the values of **private** data members.
 - *set* function is used to change the value of the data member.

3.6 Data Members, *set* Functions and *get* Functions

```
Line 25: string getCourseName()  
Line 26: {  
Line 27:     return courseName;  
Line 28: }
```

- ***get* function is used to get the value of the data member.**
- **A function that specifies a return type other than `void`**
 - **Returns a value to its calling function.**

3.6 Data Members, *set* Functions and *get* Functions

```
Line 31: void displayMessage()  
Line 32: {  
        ...  
Line 35: cout << "Welcome to the grade book for \n" << getCourseName() << "!"  
Line 36: << endl;  
Line 37: }
```

- ***set* and *get* functions can be used by other member functions of the same class.**

3.6 Data Members, *set* Functions and *get* Functions

```
Line 49: cout << "Initial course name is: " << myGradeBook.getCoursename()  
Line 50: << endl;  
        ...  
Line 55: myGradeBook.setCourseName( nameOfCourse);
```

- ***set* and *get* functions can be used to modify private data outside the class definition.**

3.6 Data Members, *set* Functions and *get* Functions

- The advantages of using *set* and *get* functions: You can control other programs to access data members in a class.
- For example: an object `student` represents a student's record and the data member `marks` represents his/her examination marks.
 - Member function `getMarks` returns the grade to you only but not the marks.

```
string getMarks()
{
    if (marks >= 60)
        return "Pass";
    else
        return "Fail";
}
```

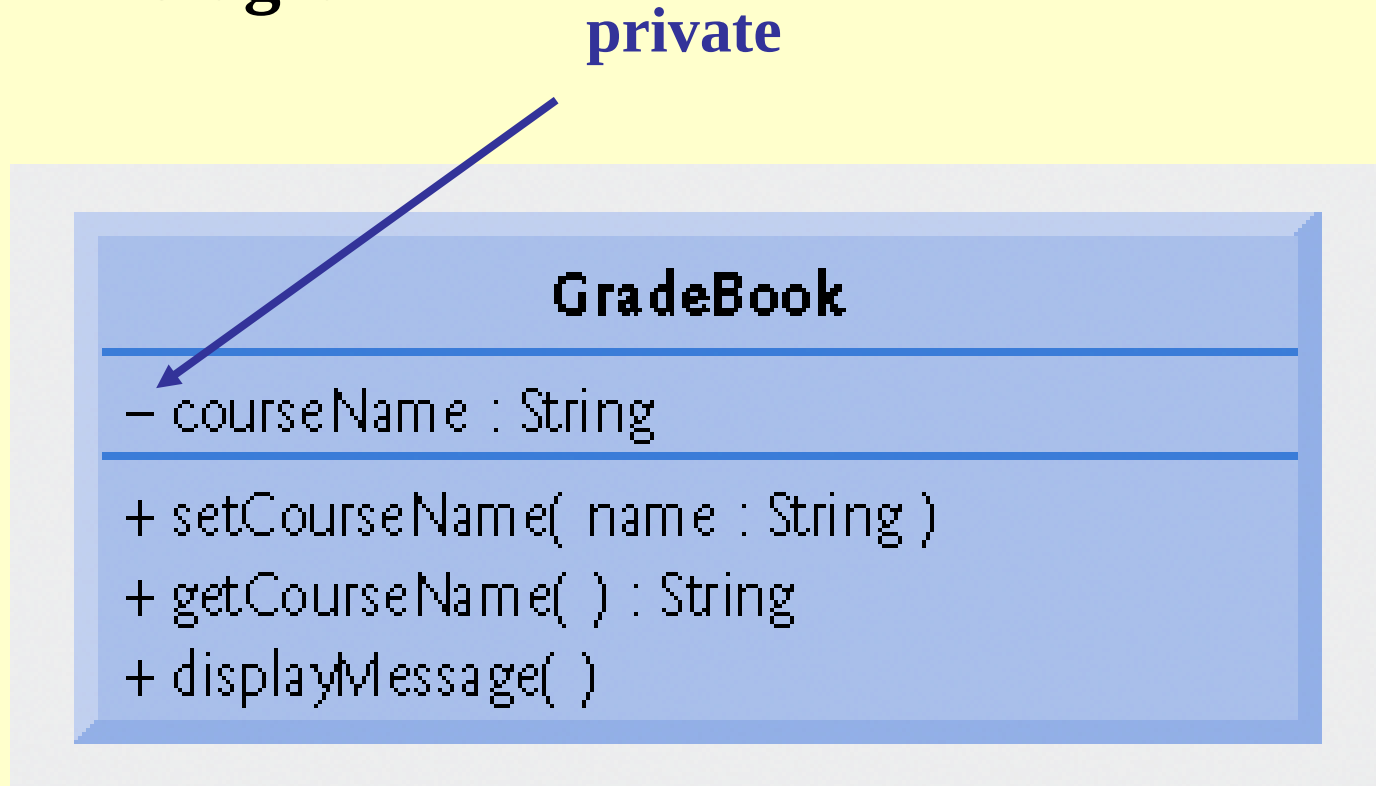
3.6 Data Members, *set* Functions and *get* Functions

- Member function **setMarks** returns nothing. The marks will be saved only if it is between 0 and 100.

```
void setMarks(int inputMarks)
{
    if ((inputMarks >= 0) && (inputMarks <= 100))
        marks = inputMarks;
}
```

3.6 Data Members, *set* Functions and *get* Functions

- UML diagram



3.7 Initializing Objects with Constructors

- **Example 4: Show you how to develop a constructor.**

Outline

fig03_07.cpp

(1 of 3)

```
1 // Fig. 3.7: fig03_07.cpp
2 // Instantiating multiple objects of the GradeBook class and using
3 // the GradeBook constructor to specify the course name
4 // when each GradeBook object is created.
5 #include <iostream>
6 using std::cout;
7 using std::endl;
8
9 #include <string> // program uses C++ standard string class
10 using std::string;
11
12 // GradeBook class definition
13 class GradeBook
14 {
15 public:
16     // constructor initializes courseName with string supplied as argument
17     GradeBook( string name )
18     {
19         setCourseName( name ); // call set function to initialize courseName
20     } // end GradeBook constructor
21
22     // function to set the course name
23     void setCourseName( string name )
24     {
25         courseName = name; // store the course name in the object
26     } // end function setCourseName
27
```

```

28 // function to get the course name
29 string getCourseName()
30 {
31     return courseName; // return object's courseName
32 } // end function getCourseName
33
34 // display a welcome message to the GradeBook user
35 void displayMessage()
36 {
37     // call getCourseName to get the courseName
38     cout << "Welcome to the grade book for\n" << getCourseName()
39         << "!" << endl;
40 } // end function displayMessage
41 private:
42     string courseName; // course name for this GradeBook
43 }; // end class GradeBook
44

```

Outline

fig03_07.cpp

(2 of 3)


```
45 // function main begins program execution
```

```
46 int main()
```

```
47 {
```

```
48     // create two GradeBook objects
```

```
49     GradeBook gradeBook1( "CS101 Introduction to C++ Programming" );
```

```
50     GradeBook gradeBook2( "CS102 Data Structures in C++" );
```

```
51
```

```
52     // display initial value of courseName for each GradeBook
```

```
53     cout << "gradeBook1 created for course: " << gradeBook1.getCourseName()
```

```
54         << "\ngradeBook2 created for course: " << gradeBook2.getCourseName()
```

```
55         << endl;
```

```
56     return 0; // indicate successful termination
```

```
57 } // end main
```

```
gradeBook1 created for course: CS101 Introduction to C++ Programming
gradeBook2 created for course: CS102 Data Structures in C++
```

Outline

fig03_07.cpp

(3 of 3)

3.7 Initializing Objects with Constructors

Line 17: `GradeBook( string name )`

- **Line 17 has a constructor.**
 - It is a function used to initialize an object of the class when it is created.
 - The call is made implicitly when the object is created.
 - Must be defined with the same name as the class.
 - Cannot return values.
 - Not even `void`

3.7 Initializing Objects with Constructors

```
Line 19: setCourseName( name );
```

- **We specify a course name for a GradeBook object when the object is created.**

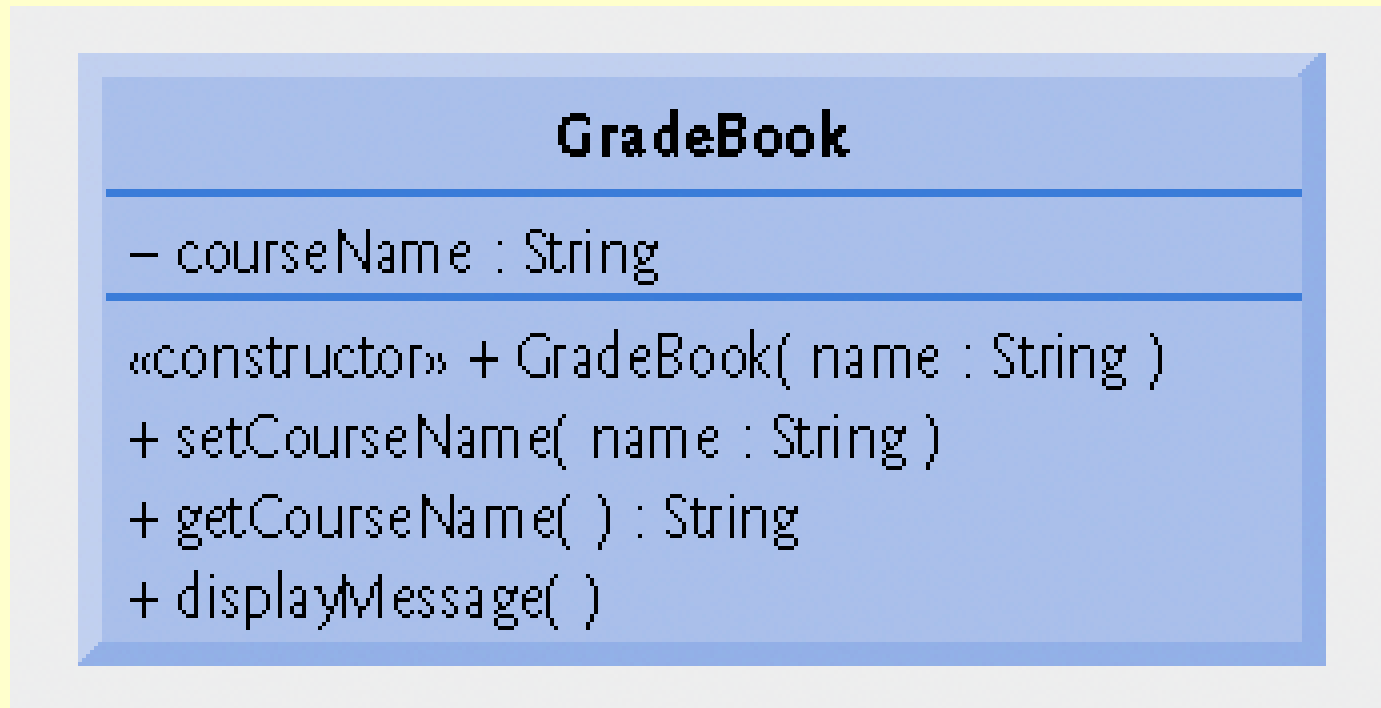
3.7 Initializing Objects with Constructors

```
Line 49: GradeBook gradeBook1( "CS101 Introduction to C++ Programming" );  
Line 50: GradeBook gradeBook2( "CS102 Data Structures in C++" );
```

- **Two objects are created with the arguments to initialize their course names.**

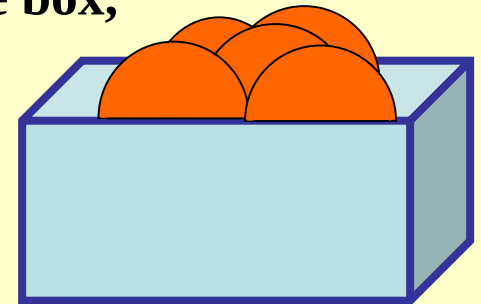
3.7 Initializing Objects with Constructors

- **Constructors in a UML class diagram**

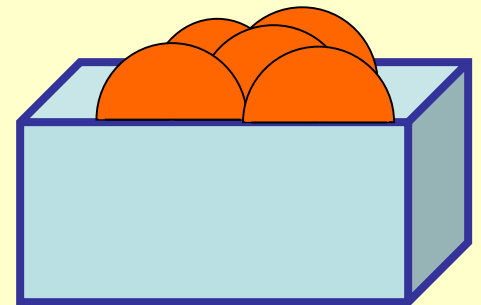


Class Work

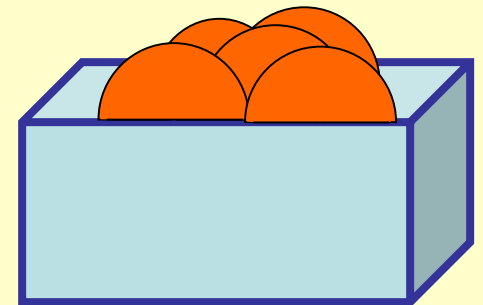
- **Write the program codes and draw an UML diagram for the class OrangeBox.**
 - A program can use it to represent a box of oranges.
 - At the beginning, there are 10 oranges in a box.
 - The class should include one data member to store the total number of oranges in the box and the data member should be accessible only to member functions of the class.
 - The class should include three member functions
 - to show the total number of oranges in the box,
 - to add some oranges to the box, and
 - to remove some oranges from the box.



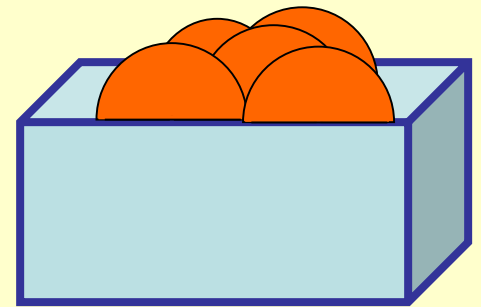
Class Work (Your work)



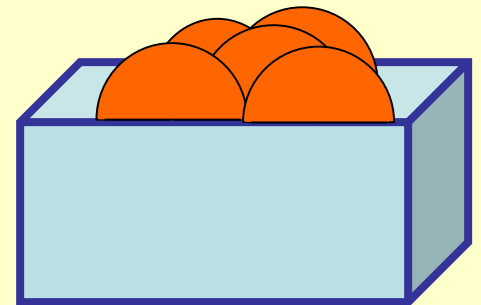
Class Work (Your work)



Class Work (Answer)



Class Work (Answer)



3.8 Placing a Class in a Separate File for Reusability

- **Example 5: To show how to place a class in a separate file.**
 - **.cpp file is known as a source-code file.**
 - **Header files**
 - **Separate files in which class definitions are placed.**
 - **Allow compiler to recognize the classes when used elsewhere.**
 - **Generally have .h filename extensions.**

3.8 Placing a Class in a Separate File for Reusability

Fig03_07.cpp

```
#include <iostream>
...
Class GradeBook
{
    ...
};

int main()
{
    ...
}
```

Class

Main program

3.8 Placing a Class in a Separate File for Reusability

GradeBook.h

```
#include <iostream>
...
Class GradeBook
{
    ...
};
```

Class

Fig03_10.cpp

```
#include "GradeBook.h"
int main()
{
    ...
}
```

Main program

```

1 // Fig. 3.9: GradeBook.h
2 // GradeBook class definition in a separate file from main.
3 #include <iostream>
4 using std::cout;
5 using std::endl;
6
7 #include <string> // class GradeBook uses C++ standard string class
8 using std::string;
9
10 // GradeBook class definition
11 class GradeBook
12 {
13 public:
14     // constructor initializes courseName with string supplied as argument
15     GradeBook( string name )
16     {
17         setCourseName( name ); // call set function to initialize courseName
18     } // end GradeBook constructor
19
20     // function to set the course name
21     void setCourseName( string name )
22     {
23         courseName = name; // store the course name in the object
24     } // end function setCourseName
25

```

Outline

GradeBook.h

(1 of 2)

```

26 // function to get the course name
27 string getCourseName()
28 {
29     return courseName; // return object's courseName
30 } // end function getCourseName
31
32 // display a welcome message to the GradeBook user
33 void displayMessage()
34 {
35     // call getCourseName to get the courseName
36     cout << "Welcome to the grade book for\n" << getCourseName()
37         << "!" << endl;
38 } // end function displayMessage
39 private:
40     string courseName; // course name for this GradeBook
41 }; // end class GradeBook

```

Outline

GradeBook.h

(2 of 2)

Outline

fig03_10.cpp

(1 of 1)

```
1 // Fig. 3.10: fig03_10.cpp
2 // Including class GradeBook from file GradeBook.h for use in main.
3 #include <iostream>
4 using std::cout;
5 using std::endl;
6
7 #include "GradeBook.h" // include definition of class GradeBook
8
9 // function main begins program execution
10 int main()
11 {
12     // create two GradeBook objects
13     GradeBook gradeBook1( "CS101 Introduction to C++ Programming" );
14     GradeBook gradeBook2( "CS102 Data Structures in C++" );
15
16     // display initial value of courseName for each GradeBook
17     cout << "gradeBook1 created for course: " << gradeBook1.getCourseName()
18         << "\ngradeBook2 created for course: " << gradeBook2.getCourseName()
19         << endl;
20     return 0; // indicate successful termination
21 } // end main
```

```
gradeBook1 created for course: CS101 Introduction to C++ Programming
gradeBook2 created for course: CS102 Data Structures in C++
```


3.8 Placing a Class in a Separate File for Reusability

Line 7: `#include "GradeBook.h"`

- **`#include` preprocessor directive**
 - Quotes indicate user-defined header files.
 - Preprocessor first looks in the current directory.
 - If the file is not found, looks in C++ Standard Library directory.
 - Angle brackets indicate C++ Standard Library (see Line 3).
 - Preprocessor looks only in C++ Standard Library directory.

3.8 Placing a Class in a Separate File for Reusability

- **Advantage: The header file can be reused by other programs.**

3.9 Separating Interface from Implementation

- **Example 6: To show how to separate the interface from the implementation.**

3.9 Separating Interface from Implementation

Fig03_07.cpp

```
#include <iostream>
...
Class GradeBook
{
    ...
};

int main()
{
    ...
}
```

Class

Main program

3.9 Separating Interface from Implementation

GradeBook.h

```
#include <iostream>
...
Class GradeBook
{
    ...
};
```

Class

Fig03_10.cpp

```
#include "GradeBook.h"
int main()
{
    ...
}
```

Main program

3.9 Separating Interface from Implementation

GradeBook.h

```
Class GradeBook  
{  
    ...  
};
```

Interface of the class

GradeBook.cpp

```
xxx GradeBook::yyy  
{  
    ...  
}
```

The implementation (content) of the class

Fig03_13.cpp

```
int main()  
{  
    ...  
}
```

Main program

```

1 // Fig. 3.11: GradeBook.h
2 // GradeBook class definition. This file presents GradeBook's public
3 // interface without revealing the implementations of GradeBook's member
4 // functions, which are defined in GradeBook.cpp.
5 #include <string> // class GradeBook uses C++ standard string class
6 using std::string;
7
8 // GradeBook class definition
9 class GradeBook
10 {
11 public:
12     GradeBook( string ); // constructor that initializes courseName
13     void setCourseName( string ); // function that sets the course name
14     string getCourseName(); // function that gets the course name
15     void displayMessage(); // function that displays a welcome message
16 private:
17     string courseName; // course name for this GradeBook
18 }; // end class GradeBook

```

Outline

GradeBook.h

(1 of 1)

3.9 Separating Interface from Implementation

```
Line 12: GradeBook( string );  
Line 13: void setCourseName( string );  
Line 14: string getCourseName();  
Line 15: void displayMessage();  
      ...  
Line 17: string courseName;
```

- **The interface contains data members and member function prototypes.**
 - **Function prototypes is a declaration of a function that tells the compiler the function's name, its return type and the types of its parameter.**


```

1 // Fig. 3.12: GradeBook.cpp
2 // GradeBook member-function definitions. This file contains
3 // implementations of the member functions prototyped in GradeBook.h.
4 #include <iostream>
5 using std::cout;
6 using std::endl;
7
8 #include "GradeBook.h" // include definition of class GradeBook
9
10 // constructor initializes courseName with string supplied as argument
11 GradeBook::GradeBook( string name )
12 {
13     setCourseName( name ); // call set function to initialize courseName
14 } // end GradeBook constructor
15
16 // function to set the course name
17 void GradeBook::setCourseName( string name )
18 {
19     courseName = name; // store the course name in the object
20 } // end function setCourseName
21

```

Outline

GradeBook.cpp

(1 of 2)

```

22 // function to get the course name
23 string GradeBook::getCourseName()
24 {
25     return courseName; // return object's courseName
26 } // end function getCourseName
27
28 // display a welcome message to the GradeBook user
29 void GradeBook::displayMessage()
30 {
31     // call getCourseName to get the courseName
32     cout << "Welcome to the grade book for\n" << getCourseName()
33         << "!" << endl;
34 } // end function displayMessage

```

Outline

GradeBook.cpp

(2 of 2)

3.9 Separating Interface from Implementation

Line 8: `#include "GradeBook.h"`

- **GradeBook implementation is placed in a separated source-code file (i.e., `GradeBook.cpp`).**
- **Include the header file to access the class `GradeBook`.**

3.9 Separating Interface from Implementation

```
Line 11: GradeBook::GradeBook( string name )
```

- **Binary scope resolution operator (::) ties a function to its class.**

```
1 // Fig. 3.13: fig03_13.cpp
2 // GradeBook class demonstration after separating
3 // its interface from its implementation.
4 #include <iostream>
5 using std::cout;
6 using std::endl;
7
8 #include "Gradebook.h" // include definition of class GradeBook
9
10 // function main begins program execution
11 int main()
12 {
13     // create two GradeBook objects
14     GradeBook gradeBook1( "CS101 Introduction to C++ Programming" );
15     GradeBook gradeBook2( "CS102 Data Structures in C++" );
16
17     // display initial value of courseName for each GradeBook
18     cout << "gradeBook1 created for course: " << gradeBook1.getCourseName()
19         << "\ngradeBook2 created for course: " << gradeBook2.getCourseName()
20         << endl;
21     return 0; // indicate successful termination
22 } // end main
```

```
gradeBook1 created for course: CS101 Introduction to C++ Programming
gradeBook2 created for course: CS102 Data Structures in C++
```

3.9 Separating Interface from Implementation

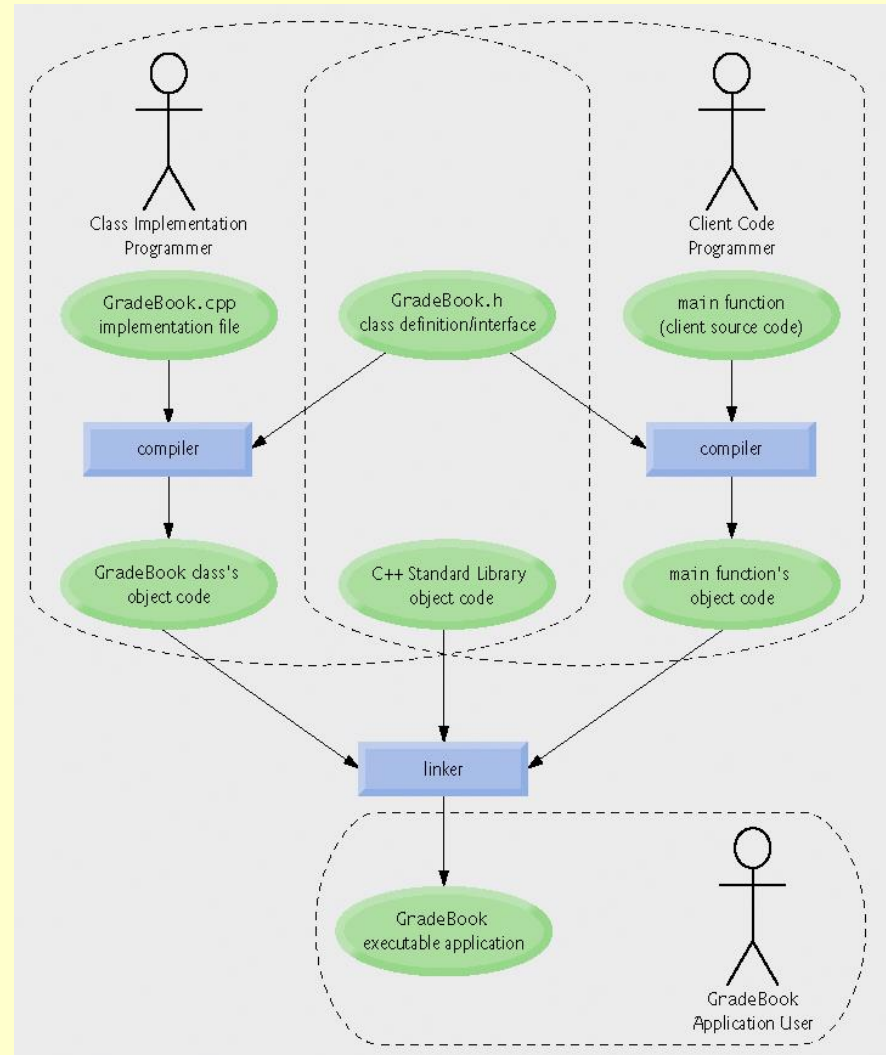
Line 8: `#include "GradeBook.h"`

- **Include the header file to access the class GradeBook.**

3.9 Separating Interface from Implementation

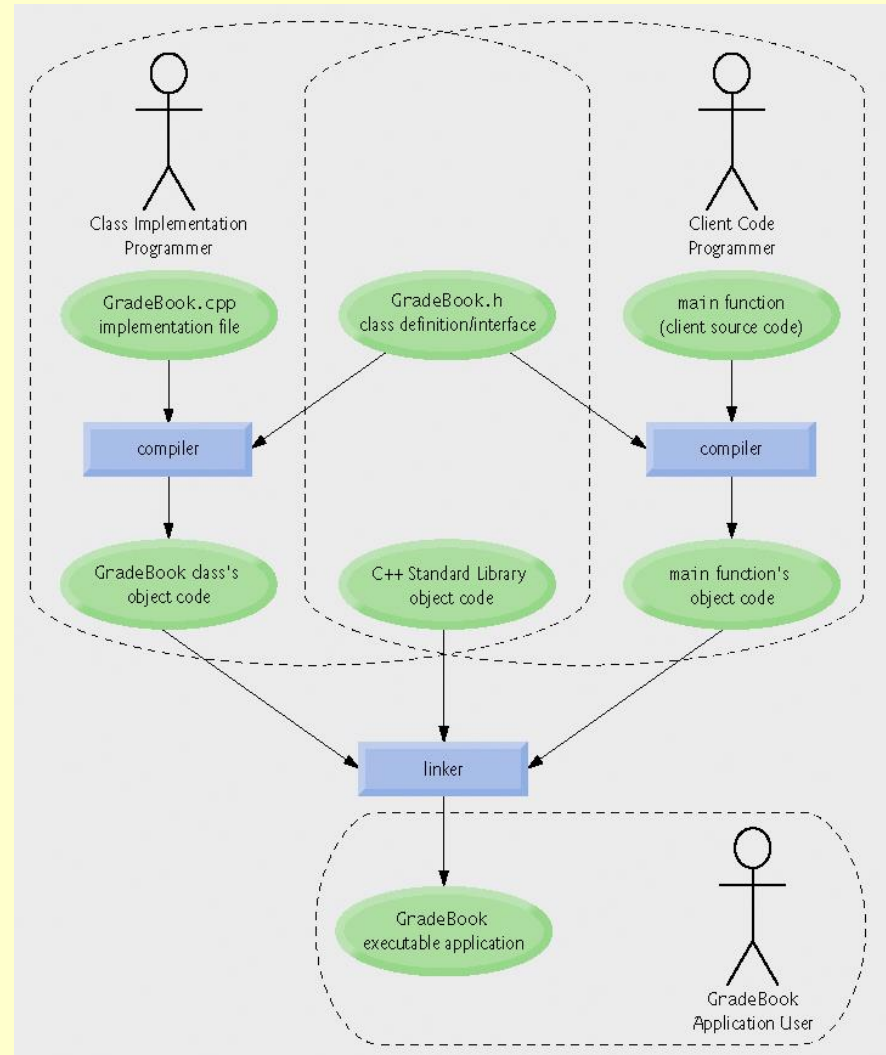
- **The Compilation and Linking Process**

- **GradeBook.h + GradeBook.cpp $\Rightarrow$ GradeBook.lib (GradeBook class's object code)**
- **GradeBook.h + fig03_13.cpp $\Rightarrow$ main function's object code**



3.9 Separating Interface from Implementation

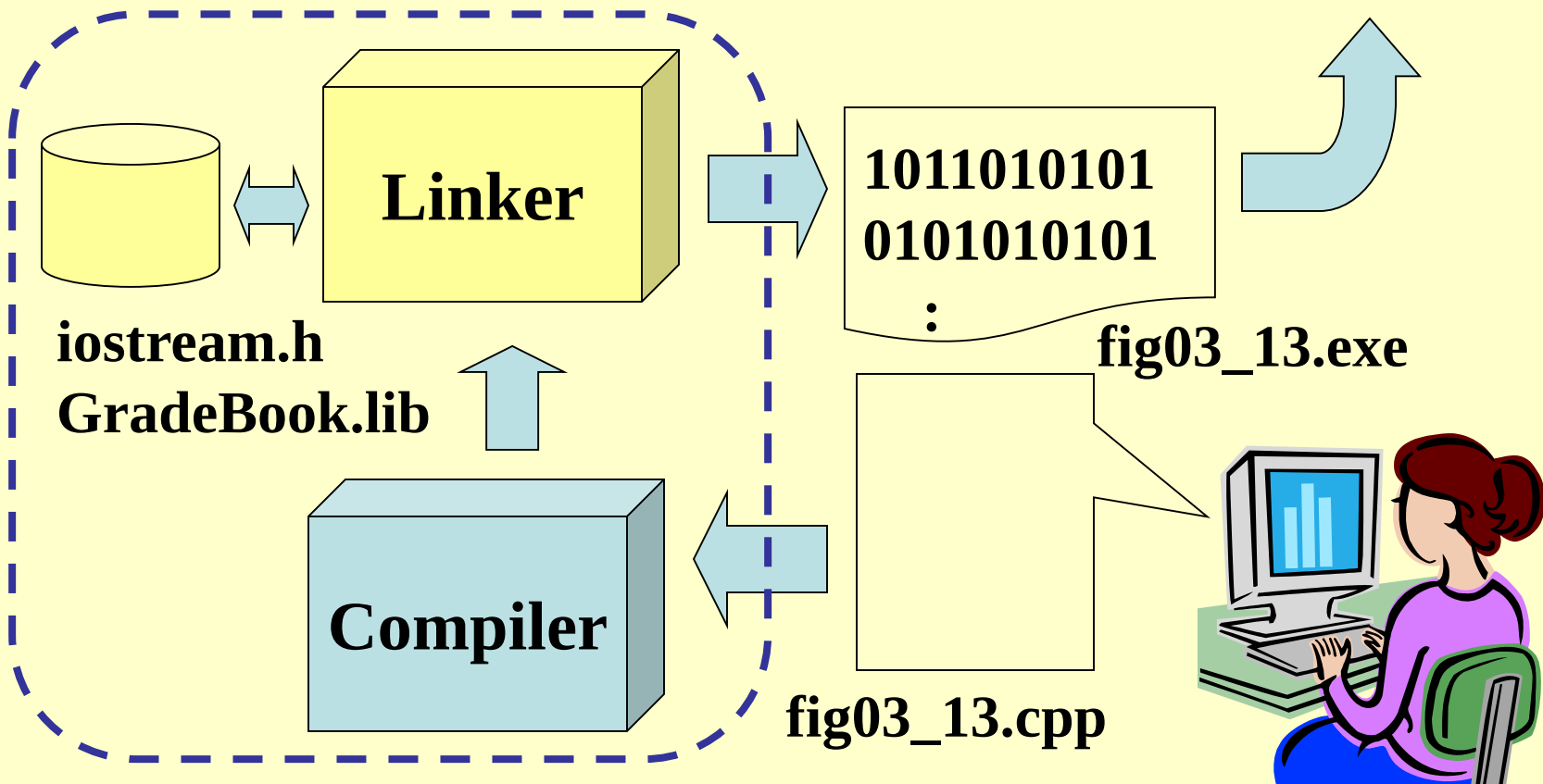
- **GradeBook class's object code + main function's object code $\Rightarrow$ GradeBook executable application**



3.9 Separating Interface from Implementation

```
C:\Windows\system32\cmd.exe
gradeBook1 created for course: CS101 Introduction to C++ Programming
gradeBook2 created for course: CS102 Data Structures in C++
Press any key to continue . . .
```

Microsoft Visual Studio



3.9 Separating Interface from Implementation

- **Advantage: Implementation details are hidden.**
 - Client code does not need to know the implementation (e.g., `GradeBook.cpp`).
 - It is good enough for the client programmer to know the interface (e.g., `GradeBook.h`) and have the object code of the implementation (e.g., `GradeBook.lib`).

3.9 Separating Interface from Implementation

- For example, the client programmer knows `GradeBook.h` such that he/she can use the member function `setCourseName` in his/her program. But he/she does not need to know the program code of the member function `setCourseName`.
- For example, we include the header file `iostream.h` such that we can use its member function `std::cout` to display messages on the screen. But we do not need to know the program code of the member function `std::cout`.

Until Next Time

- **Review Chapter 3**
- **Preview Chapter 4**